

0.97. Une lettre qui tire — A lance le tir

Le 0.96, mais le tir ne part que quand on appuie sur le bouton A, juste à droite de la lettre A. Une variable vole dit s'il est en vol : un seul tir à la fois.

1. Ce qu'il faut comprendre

C'est le 0.96, avec une chose en plus : le bouton A. Au départ, on ne voit que la lettre A, en (2, 8). Appuie sur le bouton **A** : le tir part juste à côté d'elle, en (3, 8), et file vers la droite. Quand il a disparu, on peut en relancer un.

Ce qui est nouveau ici : `vole`. Une variable qui vaut **1** quand un tir est en vol, **0** sinon. Au 0.96, on savait que le tir volait parce que `tx < 20`. Mais maintenant, au départ, il n'y a **pas** de tir : il faut le dire avec une variable à part.

Lancer : `if (bouton(A) && vole == 0)`. `&&` veut dire « et » : A est enfoncé **et** aucun tir ne vole. Alors `vole = 1`, `tx = 3` (à côté de la lettre), et on dessine le tir.

Pourquoi `vole == 0` ? Sans lui, appuyer sur A pendant le vol remettrait `tx` à 3 **sans effacer** l'ancien « . » : il resterait collé sur l'écran, et le vrai tir repartirait de la lettre. Avec lui, c'est simple : **un seul tir à la fois**. Tant qu'il vole, A ne fait rien.

Avancer : le même bloc qu'au 0.96, mais dans `if (vole == 1)`. Et quand le tir sort (`tx` arrive à 20), `vole = 0` : A peut relancer.

else : `if (tx < 20) { ... } else { ... }` : si le tir est encore dans l'écran, on le dessine ; **sinon**, on dit qu'il ne vole plus.

Deux « A » différents : la **lettre** A, à l'écran, c'est celle qui tire ; le **bouton** A, sur la console, c'est celui qu'on appuie. `ALPHABET[0]` est la lettre, `bouton(A)` le bouton.

Essaie : garde le bouton A enfoncé tout le temps. Un nouveau tir part dès que le précédent a disparu : une rafale, un tir à la fois.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **527 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	56 o	← la console, à l'allumage – la mise en route de la console
chaque_minuteur()	124 o	← main() – une fonction fournie par le langage
main()	126 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte – attend de pouvoir écrire une case
AdresseCase	10 o	← texte(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() – efface des cases
EcrireTexte	10 o	← texte() – écrit un texte, lettre par lettre
CopierTuiles	40 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
LireManette	37 o	← bouton() – lit les huit boutons
Tuiles	32 o	← CopierTuiles – 2 lettres sur 43
"."	1 o	← texte() – un texte gravé
"."	1 o	← texte() – un texte gravé

Lettres gravées : A.

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	x 2	36 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
effacer()	x 1	15 o sur place, dans main() + AdresseCase (10 o), EffacerCases (12 o)
poser()	x 1	6 o sur place, dans main()
bouton()	x 1	5 o sur place, dans main() + LireManette (37 o)
image()	x 1	3 o sur place, dans main() + AttendreImage (28 o)
images()	x 1	2 o sur place, dans chaque_minuteur()

Tes fonctions appelées :

chaque_minuteur()	x 1	10 o sur place, dans main()
-------------------	-----	-----------------------------

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UNE LETTRE QUI TIRE : A lance le tir ----
// Le 0.96, et le tir ne part plus tout seul : il part du A
// quand on appuie sur le bouton A.

#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <bouton>     // lit un bouton de la manette
#include <texte>      // écrit un texte à l'écran
#include <chaque>     // répond 1 toutes les n millisecondes
#include <effacer>    // efface des cases, ou tout le fond
```

2. UNE VARIABLE — TX

```
uint8_t tx = 3;           // la colonne du tir : juste à droite du A
```

3. UNE VARIABLE — VOLE

```
uint8_t vole = 0;         // NOUVEAU : 1 = le tir est en vol ; 0 = pas de tir
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    poser(2, 8, ALPHABET[0]); // la lettre A, celle qui tire : (2, 8), comme au 0.96

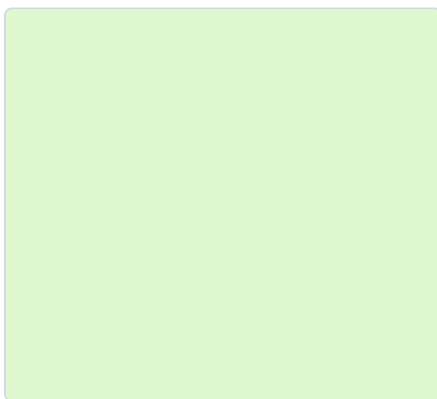
    while (true) {
        image();

        // ---- NOUVEAU : le bouton A lance le tir ----
        // bouton(A) : vrai tant que A est enfoncé.
        // && : « et ». Il faut A enfoncé ET aucun tir en vol.
        // Ainsi, UN SEUL TIR À LA FOIS : tant qu'il vole, A ne fait rien.
        if (bouton(A) && vole == 0) {
            vole = 1;           // le tir est en vol...
            tx = 3;             // ...il repart de la colonne 3, à côté du A...
            texte(tx, 8, ".");  // ...et on le dessine
        }

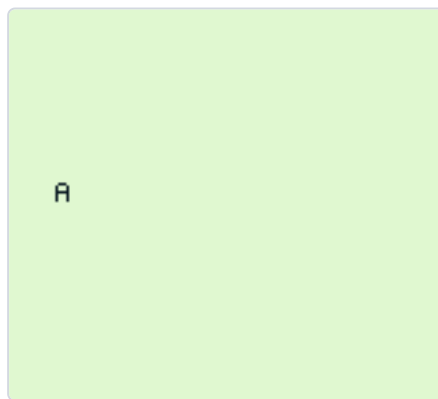
        // ---- le tir avance (le 0.96), mais seulement s'il est en vol ----
        if (chaque(100)) {
            if (vole == 1) {
                effacer(tx, 8, 1); // 1. il quitte sa case
                tx = tx + 1;        // 2. une colonne plus loin
                if (tx < 20) {      // 3. encore dans l'écran : on le dessine
                    texte(tx, 8, ".");
                } else {           // sorti par la droite :
                    vole = 0;      // plus de tir en vol, A peut en relancer un
                }
            }
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



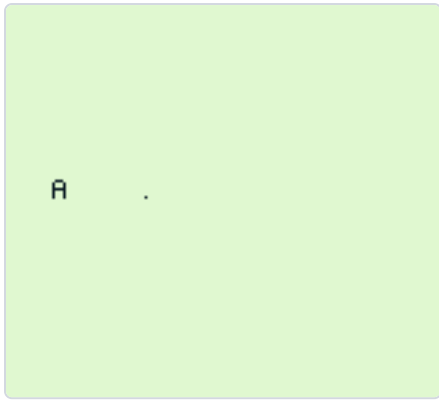
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran a changé

5. Ce qu'on doit voir

La lettre A seule en (2, 8). Le bouton A lance un « . » juste à sa droite, qui file jusqu'au bord ; tant qu'il vole, A ne fait rien ; quand il a disparu, A en relance un.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ : la lettre A en (2, 8), pas de tir
- ✓ bouton A : un « . » part juste à droite de la lettre, en (3, 8)
- ✓ A pendant le vol : rien ne change, toujours un seul tir
- ✓ sorti par la droite : vole revient à 0, il ne reste que le A
- ✓ A : un nouveau tir part

Ensuite : **0.97.1. Une lettre qui tire — A lance plusieurs tirs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++